ResumeMatch

Explainable, no-black-box candidate screening — upload a job description and a batch of resumes, get a ranked shortlist with a visible reason for every score.

Functionally complete, tested | Not yet deployed

100%	31/31	0	6+1
ranking accuracy across 50 job domains, incl. adjacent-role confusion tests	automated tests passing (matching, scoring, queueing, isolation)	paid third-party APIs — matching runs on local, open models	curated skill domains, plus an auto-extract fallback for any role

THE PROBLEM

Recruiters can't read 200 resumes and trust their own consistency

Screening is the highest-volume, lowest-leverage part of hiring. A recruiter with 150 resumes for one role either skims (and misses good candidates) or reads every one closely (and burns a week). Most screening tools solve this with a score nobody can question — a black-box number with no way to check *why* a candidate ranked where they did.

ResumeMatch is built around the opposite bet: every score should be small enough to audit by hand. If a recruiter disagrees with a ranking, they should be able to see exactly which requirement tipped it, in one click.

HOW IT WORKS

Four steps from job description to ranked shortlist

1 **Paste the job description**

The system reads the JD and figures out what kind of role it is — engineering, sales, healthcare, a skilled trade, or anything else — then pulls out the specific skills and requirements that actually matter for that role.

2 **Upload resumes, any volume**

Up to 100 resumes at once (PDF or Word). Each one is parsed and scored in the background, with a live progress bar — the recruiter doesn't wait on one giant blocking request.

3 **Every resume gets two scores that add up to one**

A **semantic score** (how closely the overall resume reads like the job, in meaning, not just keywords) and a **skill score** (what fraction of required skills are actually present — including skills described in different words, not just exact keyword matches).

4 **Recruiter gets a ranked list with receipts**

Every candidate shows matched skills, missing skills, and skills the tool inferred from context — each with the actual sentence from the resume that justified it.

EXPLAINABILITY, CONCRETELY

What a recruiter actually sees per candidate

This is the core idea the whole tool is built around — not a summary score, a breakdown.

Jordan Reyes — Service Delivery Lead 79.6%

MATCHED — STATED DIRECTLY

SLA compliance | Incident management | Vendor management

INFERRED — DESCRIBED, NOT NAMED

ITIL v4

MISSING

KPI reporting

Evidence for "ITIL v4": "Delivered weekly KPI reporting to senior stakeholders and led root cause analysis for recurring incidents" — never says ITIL by name, but the process matches closely enough to count.

WHAT IT CAN SCREEN FOR

Not just tech roles

Most JD-matching tools are secretly tech-recruiting tools with a generic label — they know what "Kubernetes" is and nothing else. ResumeMatch was deliberately tested against 50 non-tech professions (chef, veterinarian, notary public, electrician, midwife, dog trainer, and more), each scored against a resume from an adjacent-but-wrong profession as a stress test. It correctly ranked the right-fit resume higher in all 50 cases.

Curated domains

Tech, service delivery, sales, marketing, finance & accounting, and HR & recruiting each have a hand-built list of real skills and tools for that field.

Everything else

For any other role, the system pulls the actual requirement phrases straight out of that JD and screens against those instead — no pre-built list required.

Multi-tenant by design

Every company's HR team signs in with their own work email and only ever sees their own jobs and candidates — verified with real cross-company access attempts, all blocked.

No black-box AI cost

Matching runs on open embedding models hosted locally — no per-resume API bill, no sending candidate data to a third-party AI vendor.

HONEST STATUS

What's solid, and what isn't live yet

The product itself — matching, ranking, multi-tenancy — is built and tested. What's missing is standard "ready for strangers" infrastructure, not core functionality.

- **Not yet deployed**
  Currently runs locally only. Needs a real host (a persistent server, not serverless) before anyone outside this machine can use it.

- **Email delivery**
  Account sign-up emails currently go through a rate-limited default provider — needs a real email service connected before public sign-ups will work reliably.

- **No monitoring yet**
  Errors currently only show up in server logs — no alerting if something breaks in production.

- **No stated data policy**
  Resumes contain real personal information; a retention and deletion policy needs to be written before real candidate data is stored at scale.

FOR TECHNICAL REVIEWERS

Stack

Node.js/Express API, React/Vite frontend, Supabase (Postgres + storage + auth). Resume parsing via `pdf-parse` / `mammoth`. Embeddings via `@xenova/transformers` running two local models: MiniLM for the persisted semantic score, MPNet for transient per-skill paraphrase matching. Vitest for automated tests.

`Node.js + Express` `React + Vite` `Supabase (Postgres, Auth, Storage)` `Local embedding models` `Tailwind CSS` `Vitest`